

Visual surveillance of on-street parking spaces

Computer Vision : Assignment 2

Popescu Ioana-Livia
407

June 25, 2024

1 Introduction

The following project presents an automatic identification and tracking system for the surveillance of on-street parking spaces. The approach for object detection and tracking is mainly based on the YOLOv8 implementation provided by Ultralytics and manual selection of desired areas to be analyzed, adjusted based on the training data.

2 Approach

2.1 Task1

2.1.1 Task description

Task 1 asked for on-street parking spaces as being occupied or not given a video frame classification system. Thus, given a video frame and a query text file (that contains the number of parking lots to be checked on the first line and each id of the corresponding parking lot on the next lines), a ground truth file that associates each id with a value 1 - occupied - or 0 - not occupied - should be created.

2.1.2 Solution

The final solution consists in using *YOLO("yolov8n.pt")* only for classes 2 (car) and 7 (truck) in order to detect cars and trucks (upon testing, I noticed that some cars were classified as *truck*) with *conf = 0.1* (a very low value for the confidence threshold has been chosen as higher values did not perform well upon testing, failing to detect certain cars or trucks). Upon obtaining the corresponding bounding boxes for the identified objects, the center of the bounding box is computed. Next, if the center is inside any of the polygons that describe the selected parking lots, the parking lot will be considered to be occupied.

2.1.3 Object detection

Upon obtaining the bounding boxes of the identified objects using *YOLO("yolov8n.pt")*, the initial approach was to check if the bounding box of the object had an iou higher than a specific threshold with the area of a specific parking lot. Although this approach scored perfectly on Task 1, it was abandoned when moving to Task 2, as it failed for the case where parking lots towards the back had configurations such as: **occupied not occupied occupied** - it would mark the unoccupied parking lot as occupied. In order to make that approach work, I thought about setting specific iou thresholds for specific parking lots, however it was rather a tedious approach.

After studying a few examples, I realized that it would be much simpler and safer to consider the center of the bounding box instead of the whole bounding box and check if it is within the parking lot area. This is the final approach for Task 1.

Another approach I initially thought of and tested consisted in cropping the parking lot areas and running the chosen model on those, however it performed poorly.

2.1.4 Parking lot area selection

The areas of the ten parking lots have been obtained by manually selecting 4 points for each area that were later used to form a polygon that would resemble the parking lot. The method implemented for the selection is *select_parking_lots* and it takes an image and a number of areas to be selected as parameters, returning a dictionary where the keys are ids of the areas based on the selection order (1,..., N) and the values are lists of the polygon's 4 corners as points, where each point is represented as a tuple (x,y).

Multiple configurations have been tested and adjusted based on the training data (mainly Task 1 and Task 2 data), and the final configuration can be viewed in the figure below 1. In order to display the result, a method *display_parking_lots* has been created, which simply takes the values of the dictionary *self.parking_lots* and displays the corresponding polygon for each area.

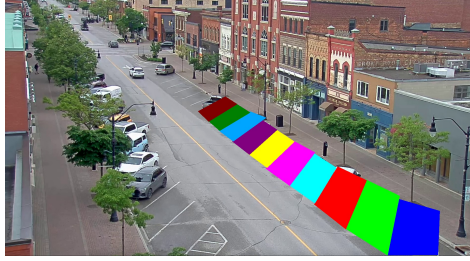


Figure 1: Parking lots areas

When testing, I used the debug feature implemented in the *detect_cars_in_parking_lots* which incorporates the whole logic for car detection within the selected parking lots for an image. In debug mode, the areas of the selected parking lots are marked with blue and the centers of the detected bounding boxes with red - thus, it was easier to see where enlarging the area of a parking lot was needed. An example where all the parking lots are displayed on the image 03.2.jpg from Task 1 can be viewed below 2.

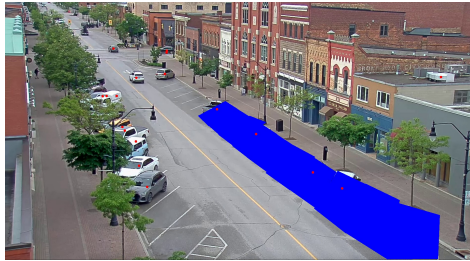


Figure 2: Parking lots areas (Debugging mode)

2.2 Task 2

2.2.1 Task description

Task 2 asked for the updated configuration of parking spaces given the initial configuration and a video stream to process.

2.2.2 Solution

Although a more thorough approach would have probably involved performing tracking of the detected cars and identifying whether they stopped in a certain parking area or not, I decided to first try to apply the solution of Task 1 on the last frame of the videos given for Task 2. The experiments were favorable, only needing some tweaks in terms of parking lot areas and model parameters, thus I decided to keep this approach, considering the ids for the selected parking areas to be $[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$ for each test (this being the default value for the *selected_lots* parameter of the *detect_cars_in_parking_lots* method).

2.2.3 Selecting the last frame

In order to save the last frame of each video of Task 2, I created the method *save_last_frames* which saves the last frame that is obtained by calling the method *get_last_frame* which goes through each frame, updating the variable *prev_frame* at each step until there is no frame left in the video - thus, the last frame will be stored in the variable *prev_frame*.

2.3 Task 3

2.3.1 Task description

Task 3 asked to track a specific vehicle in the scene containing on-street parking spaces in a given video, also being provided with the initial bounding box of the object to be tracked.

2.3.2 Solution

The final approach implied creating a method *track* that would track a given object given initial bounding box coordinates using *YOLO("yolov8x.pt")*. I decided to opt for this version of the yolo model as the one used for tasks 1 and 2 performed poorly on this task. The values for the parameters are *conf=0.1*, *iou=0.5*, *show=False*, *persist=True*, *verbose=False*, *classes = [2,7]*, *tracker=f'TRACKER_PATH'*, where *TRACKER_PATH* is the path to a custom tracker of type *bytetrack* I ended up using upon performing experiments. By setting lower values for *track_high_thresh* (threshold for the first association) and *new_track_thresh* (threshold for initiation of new track if the detection does not match any tracks), I managed to fix the issue of not identifying the desired object in a few examples from train (especially the 8th) and achieve better overall results.

2.3.3 Identify the desired object's id

The identification of the desired object's id has been tricky as in some frames the desired id was not to be found - therefore, I created a variable *discovered_track_id* that would be *None* if an id was not yet associated to our desired object (known to originally be within the initial bounding box coordinates provided). In order to match the object to the id of a detected object in the specified classes (2 and 7), I computed the iou of the initial bounding box and the detected bounding box and retained the maximum iou per frame. If *discovered_track_id* was still *None*, I would simply take the values from the previous frame as the values for the current frame as well, since we are not interested in a perfect track, but in a very close one. On the other hand, if the *discovered_track_id* was not *None*, the coordinates of the detecting bounding box for that id in the current frame would be computed and stored in *frame_coords* - a list of coordinates for each frame, based on which the output file will be built on.

2.4 Task 4

2.4.1 Task description

Task 4 asked for the system to be able to count the number of vehicles stopped at a traffic light in the scene containing on-street parking spaces in a given video.

2.4.2 Solution

As the number of vehicles stopped at a traffic light in the scene is computed based on the last frame, I considered extracting the last frame with the same method I used for Task 2 and detecting the cars waiting at the queue within a selected queue area, applying the same concept as in tasks 1 and 2. Although a more appropriate and accurate algorithm would have implied checking which objects are actually stopping at the traffic light (having the same bounding box for a specific number of frames), the approach at hand had favorable results and I decided to keep it.

2.4.3 Queue area selection

The selected queue area has been selected with the same method used for the parking areas and is displayed in the figure below 3. It is definitely not ideal as a larger queue could happen (for example

a full street queue in case of an accident), and a better mechanism that used tracking to determine if a car stopped in the queue (possibly considering checking if a car in front of it is also stopped), would have been more accurate.

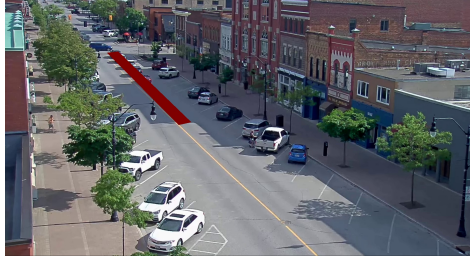


Figure 3: Queue lot

2.4.4 Object detection

For object detection I used *YOLO("yolov8x.pt")* with $conf = 0.05$, $classes = [2, 7]$. The method *detect_cars_in_queue* considers the results from the model applied on the last frame and adds the center of the detected bounding boxes in a list called *ans* if they are in the queue lot area. The difference is that duplicates appeared often, therefore I implemented an additional check based on which only certain bounding boxes will be kept. Thus, for each recorded point in *ans*, I am looking at the next ones in *ans* and check if the circles drawn with a radius of 10 are overlapping - if so, the point I was currently at will be marked to not be kept. This approach was inspired by simply implementing the visual checks I was performing and it worked quite well - I checked the centers of the bounding boxes in order to make sure each corresponds to a specific car and the number of total cars isn't a happy coincidence. An example of the center of the kept bounding boxes can be viewed in figure 4a and an examples of checking whether this centers correspond to the desired car can be viewed in figure 4b, as I considered drawing larger circles starting from the center good enough for a sanity check.



(a) Centers of kept bounding boxes



(b) Sanity check of identified center points

Figure 4: Analysis of bounding box centers of detected cars in queue lot

3 Experiments and results

The configuration has been adjusted based on training data, obtaining the following scores:

Task	Tests
1	50/50
2	15/15
3	15/15
4	15/15

Table 1: Total score per task